

Use Thread Pools Correctly: Keep Tasks Short and Nonblocking

How many threads must a thread pool pool? For a thread pool must pool threads

By Herb Sutter

Herb Sutter is a bestselling author and consultant on software development topics, and a software architect at Microsoft. He can be contacted at www.gotw.ca

What are thread pools for, and how can you use them effectively? As shown in Figure 1, thread pools are about letting the programmer express lots of pieces of independent work as a "sea" of tasks to be executed, and then running that work on a set of threads whose number is automatically chosen to spread the work across the hardware parallelism available on the machine (typically, the number of hardware cores [1]). Conceptually, this lets us execute the tasks correctly one at a time on a single-core machine, execute them faster by running four at a time on a four-core machine, and so on.

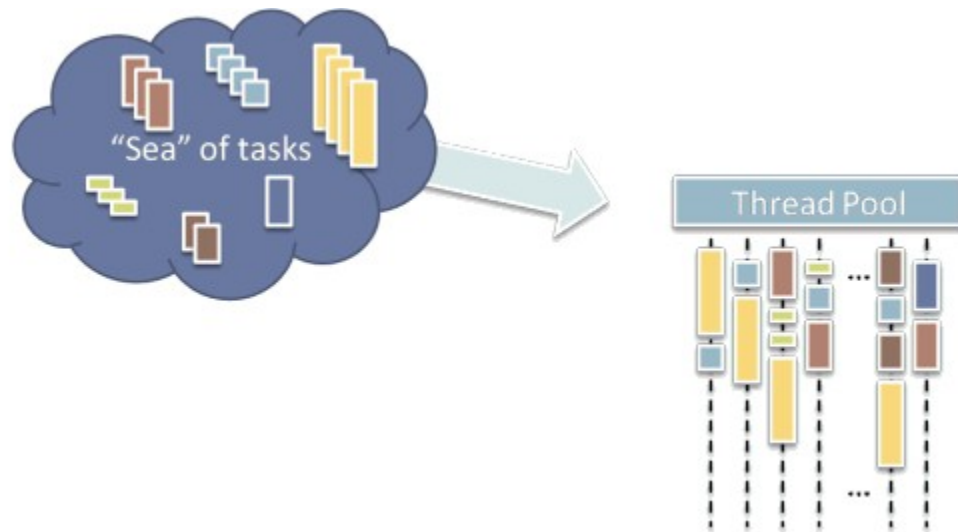


Figure 1: Thread pools are about taking a sea of work items and spreading them across available parallel hardware.

Besides scalable tasks, one other good candidate of work to run on a thread pool is the small "one-shot" thread. This is work that we might ordinarily express as a separate thread, but that is so short that the overhead of creating a thread is comparable to the work itself. Instead of creating a brand new thread and quickly throwing it away again, we can avoid the thread creation overhead by running the work on a thread pool, in effect playing "rent-a-thread" to reuse an existing pool thread instead. (See [2] for more about using threads correctly, including running small threads as pool work items.)

But the thread pool is a leaky abstraction. That is, the pool hides a lot of details from us, but to use it effectively we do need to be aware of some things a pool does under the covers so that we can avoid inadvertently hitting performance and correctness pitfalls. Here's the summary up front:

- Tasks should be small, but not too small, otherwise performance overheads will dominate.
- Tasks should avoid blocking (waiting idly for other events, including inbound messages or contested locks), otherwise the pool won't consistently utilize the

hardware well -- and, in the extreme worst case, the pool could even deadlock.

Let's see why.

Tasks Should Be Small, but Not Too Small

Thread pool tasks should be as small as possible, but no smaller.

One reason to prefer making tasks short is because short tasks can spread more evenly and thus use hardware resources well. In Figure 1, notice that we keep the full machine busy until we start to run out of work, and then we have a ragged ending as some threads complete their work sooner and sit idle while others continue working for a time. The larger the tasks, the more unwieldy the pool's workload is, and the harder it will be to spread the work evenly across the machine all the time.

On the other hand, tasks shouldn't be too short because there is a real cost to executing work as a thread pool task. Consider this code:

```
// Example 1: Running work on a thread pool.  
pool.run( [=] { SomeWork(); } );
```

By definition, **SomeWork** must be queued up in the pool and then run on a different thread than the original thread. This means we necessarily incur queuing overhead plus a context switch just to move the work to the pool. If we need to communicate an answer back to the original thread, such as through a message or Future or similar, we will incur another context switch for that. Clearly, we aren't going to want to ship **int result = int1 + int2;** over to a thread pool as a distinct task, even if it could run independently of other work. It's just like the sign you see in a theme park at the entrance to the roller coaster: "You must be at least this big to go on this ride."

So although we like to keep thread pool tasks small, a task should still be big enough to be worth the overhead of executing it on the pool. Measure the overhead of shipping an empty task on your particular thread pool implementation, and as a rule of thumb, aim to make the work you actually ship an order of magnitude larger.

Tasks Should Avoid Waiting

Next, let's consider a key implementation question: How many threads should a thread pool contain? The general idea is that we want to "rightsize" the thread pool to perfectly fit the hardware, so that we provide exactly as much work as the hardware can actually physically execute simultaneously at any given time. If we provide too little work, the machine is "undersubscribed" because we're not fully utilizing the hardware; cores are going to waste. If we provide too much work, however, the machine becomes "oversubscribed" and we end up incurring contention for CPU and memory resources, for example in the form of needless extra context switching and cache eviction. [3]

So our first answer might be:

Answer 1 (flawed): "A thread pool should have one pool thread for each hardware core."

That's a good first cut, and it's on the right track. Unfortunately, it doesn't take into account that not all threads are always ready to run. At any given time, some threads may be temporarily blocked because they are waiting for I/O, locks, messages, events, or other things to happen, and so they don't contribute to the computational workload.

Figure 2 illustrates why tasks that block don't play nice with thread pools, because they idle their pool thread. While the task is blocked, the pool thread has nothing to do and probably won't even be scheduled by the operating system. The result is that, during the time the task is blocked, we are actually providing less parallel work than the hardware could run; the machine is undersubscribed. Once the task resumes, the full load is restored, but we've

wasted the opportunity to get more work done on otherwise-idle hardware.

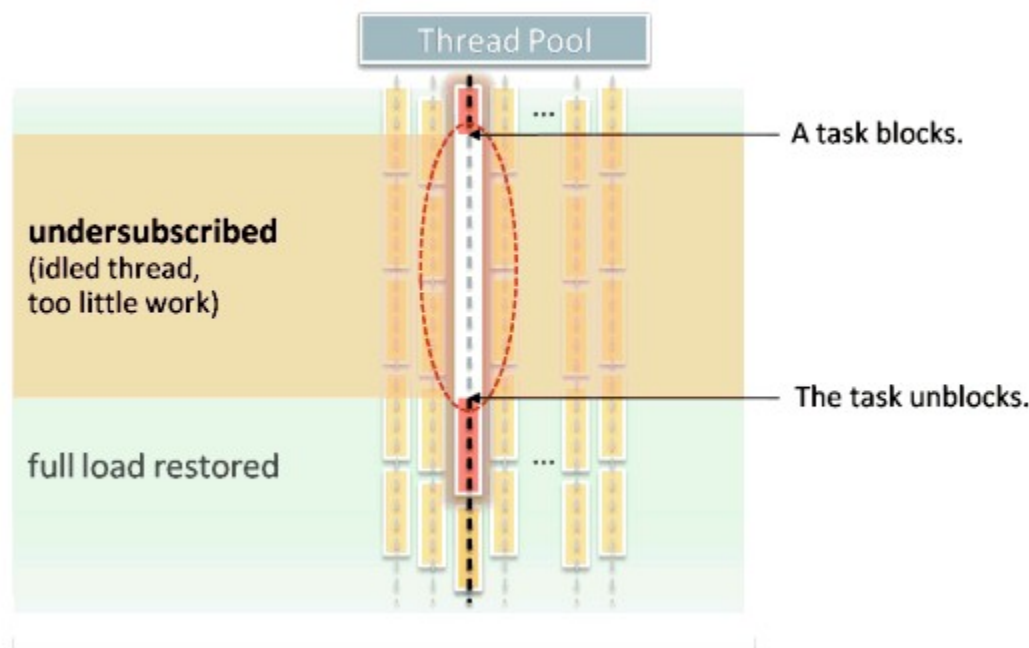


Figure 2: Blocking inside a task idles a pool thread.

At this point, someone is bound to ask the natural question: "But couldn't the pool just reuse the idled thread that's just sitting blocked anyway, and use it to run another task in the meantime?" The answer is "no," that's unsafe in general. A pool thread must run only one task at a time, and must run it to completion. Consider: The thread pool cannot reclaim a blocked thread and use it to run another task, thus interleaving tasks, because it cannot know whether the two tasks will interact badly. For example, the pool cannot know whether the blocked task was using any thread-specific state, such as using thread-local storage or currently holding a lock. It could be disastrous if the interleaved task suddenly found itself running under a lock held by the original task; that would be a fancy way to inject potential deadlocks by creating new lock acquisition orders the programmer never knew about. Similarly, the original task could easily break if it returned from a blocking call only to discover unexpected changes were made to its thread-local state made by an interloping interleaver. Some languages and platforms do provide special coroutine-like facilities (e.g., POSIX **swapcontext**, Windows **fibers**) that can let the programmer write tasks that intentionally interleave on the same thread, but these should be thought of as "manual scheduling" and "having multiple stacks per thread" rather than just one stack per thread, and the programmer has to agree up front to participate and has to opt into a restrictive programming model to do it safely. [4,5] A general-purpose pool can't safely just interleave arbitrary tasks on the same thread without the programmer's active participation and consent.

So we actually want to match, not just any threads, but specifically threads that are ready to run, with the hardware on this machine that can run them. Therefore a better answer is:

Answer 2 (better): "A thread pool should have one ready pool thread for each hardware core."

Figure 3 illustrates how a thread pool can deal with tasks that block and idle their pool thread. First, a thread pool has to be able to detect when one of its threads goes idle even though it still has a task assigned; this can require integration with the operating system. Second, once idling has been detected the pool has to create or activate an additional pool thread to take the place of the blocked one, and start assigning work to it. From the time the origin

task blocks and idles its thread to the time the new thread is active and begins executing work, the system has more cores than work that is ready to run, and is undersubscribed.

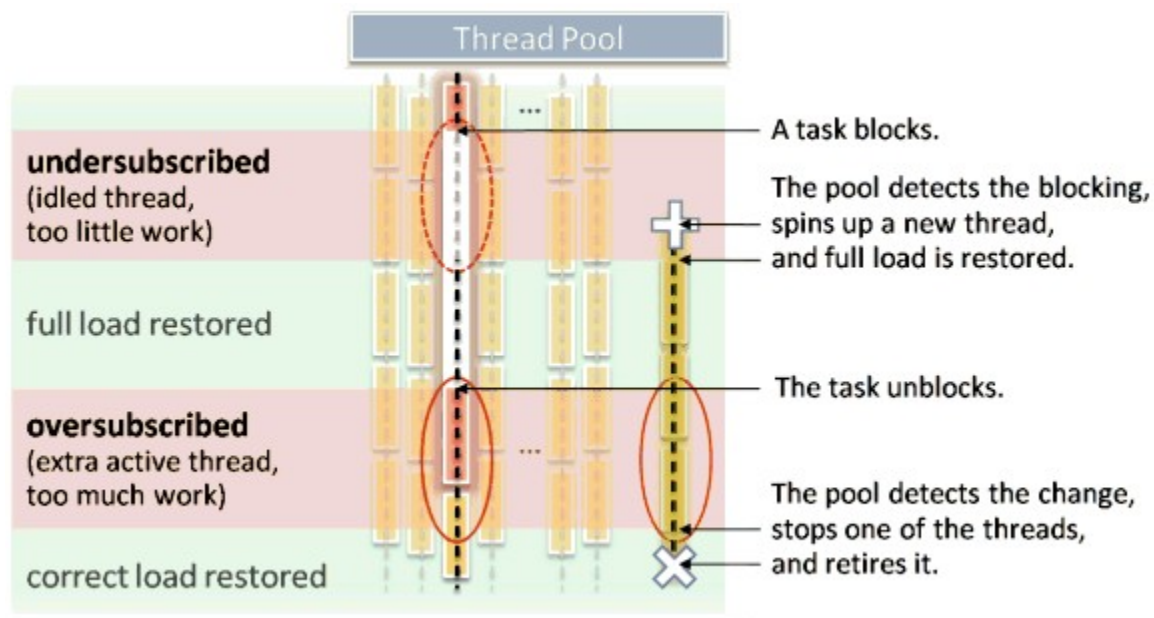


Figure 3: Thread pools can adapt by adding threads.

But what happens when the original task unblocks and resumes? Then we enter a period where the system is oversubscribed, with more work than there are cores to run the work. This is undesirable because the operating system will schedule some tasks on the same core, and those tasks will contend against each other for CPU and cache. Now we do the dance in reverse: The thread pool has to be able to detect when it has more ready threads than cores, and retire one of the threads as soon as there's an opportunity. Once that has been done, the pool is "rightsized" again to match the hardware.

The more often tasks block, the more they interfere with the thread pool's ability to match ready work with available hardware that can execute it.

Tasks Should Avoid Waiting For Each Other

The worst kind of blocking is when tasks block to wait on other tasks in the pool. To see why, consider the following code:

```
// Example 2: Launching pathologically interdependent tasks
//
// First, launch N tasks into the thread pool. Each task
// performs some work while blocking twice in the middle.
for( i = 0; i < N; ++i ) {
    pool.run( [=] {
        // do some work
        phase1[i].wait();    // A: wait point #1
        // and more work
    }
}
```

```

    phase1[i+1].signal();    // B
    // and more here
    phase2[i].wait();        // C: wait point #2
    // and still more
    phase2[i+1].signal();    // D
} );
}
// back on the parent thread
phase1[0].signal(); // release phase 1
phase1[N].wait();   // wait for phase 1 to complete
// E: what is the state of the system at this point?
phase2[0].signal(); // release phase 2
phase2[N].wait();   // wait for phase 2 to complete

```

This code launches **N** tasks into the pool. Each task performs work, much of which can execute in parallel. For example, there is no reason why the first "do some work" section of all **N** tasks couldn't run at the same time because those pieces of work are independent and not subject to any mutual synchronization.

There is some mutual synchronization, though. Each task has two points where it waits for the previous task—in this toy example it's just a simple wait; but in real code, this can happen when the task needs to wait for an intermediate result, for a synchronization barrier between processing stages, or for some other purpose. Here, there are two phases of processing. In each phase, each i^{th} task waits to be signaled, does more work, then signals the following $i+1^{\text{th}}$ task.

Execution proceeds as follows: After launching the tasks, the parent thread kicks them past the first wait simply by signaling the first task. The first task can then proceed to do more work, and signal the next task, and so on until the last task signals the parent that phase 1 is now complete. The parent then signals the first task to wake it up from its second wait, and the phase 2 wakeups proceed similarly. Finally, all tasks are complete and the parent is notified of the "join" and continues onward.

Now stop for a moment and consider these questions:

- What is the state of the system when we reach line E?
- How many threads must the thread pool have to execute this program correctly? What happens if it doesn't have enough threads?

Let's consider the answers.

First, the state of the system at line **E** is that we have **N** tasks each of which is partway through its execution. All **N** tasks must have started, because **phase1[N]** has been signaled by the last task, which can only happen if the previous task signaled **phase 1[N-1]**, and so on. Therefore, each of the **N** tasks must have performed its line **B**, and is either still performing the processing between **B** and **C**, or else is waiting at line **C**. (No task can be past **C** because the parent thread hasn't kicked off the phase 2 signal cascade yet.)

So, then, how many threads must the thread pool have to execute this program? The answer is that it must have at least **N** threads, because we know there is a point (line **E**) at which every one of the **N** tasks must have started running and therefore each must be assigned to its own pool thread. And there's the rub: If the thread pool has fewer than **N** threads and cannot create any more, typically because a maximum size has been reached, the program will deadlock because it cannot make progress unless all tasks are running.

Just make **N** sufficiently large, and you can deadlock on most production thread pools. For example, .NET's **ThreadPool.GetMaxThreads** returns the maximum number of threads available; the current default is 250 threads per core, so to inject deadlock on a default-configured pool, let **N** be **250 × #cores + 1**. Similarly, Java's **ThreadPoolExecutor** lets you set the maximum number of pool threads via a constructor parameter or

setMaximumPoolSize; the pool may also fail to grow if the active **ThreadFactory** fails to create a new thread by returning null from **newThread**.

At this point you may legitimately wonder: "But isn't Example 2 pathological in the first place? It even says so in the comment!" Actually, Example 2 is fairly typical of many kinds of concurrent algorithms that can proceed in parallel much of the time but occasionally need synchronization points as barriers to exchange intermediate results or enter a new stage of processing. The only thing that's pathological about Example 2 is that the program is trying to run each work item as a thread pool task; instead, each work item should run on its own thread, and everything would be fine.

Summary

Thread pool tasks should be as small as possible, but no smaller. The shorter the tasks, the more evenly they will spread across the pool and the machine, but the more the per-task overhead will start to dominate. Thread pool tasks should still be big enough to be worth the round-trip overhead of shipping them off to a pool thread to execute and shipping the result back, without having the overhead dominate the work itself. Thread pool tasks should also avoid blocking, or waiting for other things to happen, because blocking interferes with the ability of the pool to keep the right number of ready threads available to match the number of cores. Tasks should especially avoid blocking to wait for other tasks in the pool, because when that happens it can lead to worse performance penalties -- and, in the extreme worst case, having a large number of tasks that block waiting for each other can deadlock the entire pool, and with it your application.

Notes

[1] I'll use the term "cores" for convenience to denote the hardware parallelism. Some CPUs have more than one hardware thread per core, and so the **actual amount of hardware parallelism available = #processors × #cores/processor × #hardware threads/core**. But "total number of hardware threads" is a mouthful, not to mention potentially confusing with software threads, so for now I'll stick with talking about the "total number of cores."

[2] H. Sutter. ["Use Threads Correctly = Isolation + Asynchronous Messages"](#) (*Dr. Dobb's Digest*, April 2009).

[3] H. Sutter. ["Sharing Is the Root of All Contention"](#) (*Dr. Dobb's Digest*, March 2009).

[4] [Single UNIX Specification](#) (IEEE Standard 1003.1, 2004).

[5] [Windows Fibers \(MSDN\)](#).